I numeri reali: virgola mobile

Unità di Apprendimento 3 · Lezione 3

Tecnologie e Progettazione di Sistemi Informatici e di Telecomunicazioni (TPSI)

Classe 3ª · Istituto Tecnico – Informatica

Prof. Dicara

In questa lezione impareremo

- a rappresentare i **numeri reali** in un calcolatore con la **virgola mobile** (*floating point*);
- mantissa, esponente, **normalizzazione** e **bit nascosto** (*hidden bit*);
- lo standard **IEEE-754** a 32, 64 e 80 bit;
- a **codificare** e **decodificare** un numero reale;
- **underflow**, numeri denormalizzati, ±0, ±∞, NaN;
- **errore** di rappresentazione: troncamento e arrotondamento.

Mappa della lezione

Parte	Argomento
A	Mantissa ed esponente
B	Normalizzazione e hidden bit
C	Esponente in eccesso (bias)
D	Standard IEEE-754: codifica
E	Decodifica
F	Casi speciali e denormalizzati
G	Errori: troncamento, arrotondamento

Parte A

Mantissa ed esponente

Perché serve la virgola mobile

Con la virgola **fissa** si ha un numero prefissato di cifre intere e decimali: non si possono rappresentare insieme numeri enormi (massa del Sole: $2 \cdot 10^{30}$ kg) e piccolissimi (massa dell'elettrone: $9 \cdot 10^{-31}$ kg).

La **notazione scientifica** risolve il problema: **numero = mantissa × base^{esponente}**.

Numero	Notazione scientifica
1 234 000	$1{,}234 \cdot 10^{6}$
0,00056	$5{,}6 \cdot 10^{-4}$

La **virgola "si sposta"** (mobile) a seconda dell'esponente.

Mantissa, esponente e resto

- una **mantissa m** (reale) di esattamente n cifre
- un **resto r** (reale)
- un **esponente esp** (intero)

$$N = m \cdot B^{esp} + r$$

Un numero reale *N* si scrive come **N ≈ m · B^esp**, con *m* = **mantissa**, *B* = **base**, *esp* = **esponente**. Il **resto** *r* è la parte che non entra nelle cifre disponibili ed è **trascurata**.

> **Precisazione:** la scrittura esatta è N = (m + r) · B^esp, dove *r* è la parte della mantissa non rappresentabile con le cifre disponibili.

Infinite scomposizioni

> Esistono **infinite triple** (**m**, **r**, **esp**) in grado di rappresentare nella stessa base **B**, a parità di **n**, lo stesso valore reale.

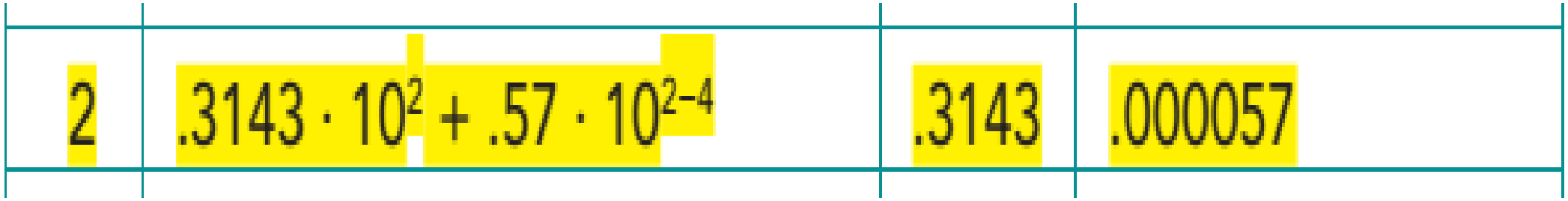
Lo stesso numero ha **infinite scomposizioni** (mantissa, esponente) diverse:

$12{,}5 = 1{,}25 \cdot 10^{1} = 12{,}5 \cdot 10^{0} = 0{,}125 \cdot 10^{2} = 125 \cdot 10^{-1}$ ...

Serve una **regola** per sceglierne una sola: la **normalizzazione**.

Tabella delle scomposizioni

esp	scomposizione	m	r
−1	$314.3 \cdot 10^{-1} + 570 \cdot 10^{-1-4}$	314.3	.057
0	$31.43 \cdot 10^{0} + 57 \cdot 10^{0-4}$	31.43	.0057
1	$3.143 \cdot 10^{1} + 5.7 \cdot 10^{1-4}$	3.143	.00057
2	$.3143 \cdot 10^{2} + .57 \cdot 10^{2-4}$	.3143	.000057
3	$.0314 \cdot 10^{3} + .357 \cdot 10^{3-4}$	.0314	.00000357
4	$.0031 \cdot 10^{4} + .4357 \cdot 10^{4-4}$	.0031	.0000004357
5	$.0003 \cdot 10^{5} + .14357 \cdot 10^{5-4}$	.0003	.000000014357
6	$.0000 \cdot 10^{6} + .314357 \cdot 10^{6-4}$	.0000	.00000000314357



Tra tutte le scomposizioni se ne sceglie **una sola**, la **normalizzata**.

Parte B

Normalizzazione e hidden bit

Il numero normalizzato

$$-0.3143\overline{570} \cdot 10^{2}$$

Un numero è **normalizzato** quando la mantissa ha **una sola cifra diversa da 0 prima della virgola**.
In base 2 l'unica cifra diversa da 0 è **1**: la mantissa è sempre **1,xxxx**.

ESEMPI

Esempio 1

Codifichiamo il numero +45.6768 e otteniamo:

- con esp = 2: +45.6768 = +0.45676 · 10^2 + resto 0.0008, che **è trascurabile**;
- con esp = 4: +45.6768 = +0.00456 · 10^4 + resto 0.0768, che **non è trascurabile**;
- con esp = 6: +45.6768 = +0.00004 · 10^6 + resto 5.6768, che è praticamente tutto il numero che dovevamo rappresentare.

Esempio 2

Codifichiamo il numero +532.4321 e otteniamo:

- con esp = 3: +532.4321 = +0.53243 · 10^3 + resto 0.0021, che **è trascurabile**;
- con esp = 5: +532.4321 = +0.00532 · 10^5 + resto 0.4321, che **non è trascurabile**;
- con esp = 7: +532.4321 = +0.00005 · 10^7 + resto 32.4321 (idem come sopra).

Formule della virgola mobile

$$N = m \cdot B^{esp}$$

$$N = \text{mantissa} \cdot 2^{\text{esponente}}$$

In **base 2**: $N = \pm 1{,}f \times 2^{esp}$, dove **f** è la **parte frazionaria** della mantissa.

La struttura del numero in memoria



- S_M = il segno della mantissa;
- n_M = il numero di bit riservati alla mantissa;
- mantissa = la codifica binaria della mantissa;
- S_E = il segno dell'esponente;
- n_E = il numero di bit riservati all'esponente;
- esponente = la codifica binaria dell'esponente

Il numero viene memorizzato in tre campi:

Campo	Contenuto
S	segno (0 = +, 1 = −)
E	esponente
M	parte frazionaria della mantissa

Variante della struttura



Sia la mantissa che l'esponente hanno un numero finito di cifre e quindi gli intervalli rappresentati saranno sempre limitati, in funzione del numero dei bit (32, 64 ecc.); in ogni caso, avremo degli errori dovuti all'arrotondamento.

Il formato più utilizzato, che useremo negli esempi che seguono, è quello che **normalizza la mantissa** in modo da avere **un solo bit uguale a 1** a sinistra della virgola.

Il numero di bit di **M** limita la **precisione** (quante cifre significative); il numero di bit di **E** limita il **range** (quanto grande o piccolo).

Normalizzare passo passo

Esempio: $101{,}111_2$ (5,875)

$$101.111_2$$

Sposto la virgola di **2 posizioni a sinistra**: $1{,}01111 \times 2^2$.

$$1.01111 \cdot 2^2$$

Esempio: $0{,}001011_2$ → sposto di 3 a destra: $1{,}011 \times 2^{-3}$.

$$0.001011_2 \qquad 1.011 \cdot 2^{-3}$$

Normalizzare: un altro esempio

`100,100₂` (4,5) → `1,00100 × 2²`

$$100100.0_2 \qquad 1.00100 \cdot 2^5$$

Regola pratica:

- virgola spostata **a sinistra** di *k* posti → esponente **+ k**;
- virgola spostata **a destra** di *k* posti → esponente **− k**.

Hidden bit

Poiché la mantissa normalizzata comincia **sempre con 1**, quell'**1 non si memorizza**: è il **bit nascosto** (*hidden bit*). Così si guadagna **1 bit di precisione** gratis.

	A) prima cifra significativa diversa da 0 alla **sinistra** del punto di radice:	B) prima cifra significativa diversa da 0 alla **destra** del punto di radice:
senza bit nascosto	$N = 1.xxxxxxxx \cdot 2^{esp}$	$N = 0.1xxxxxxxx \cdot 2^{esp}$
con bit **nascosto**	$N = \cancel{1}.xxxxxxxx \cdot 2^{esp}$	$N = 0.\cancel{1}xxxxxxxx \cdot 2^{esp}$

> Con 23 bit memorizzati la precisione effettiva è di **24 bit** (23 + hidden bit).

Parte C

Esponente in eccesso

L'esponente in eccesso P

L'esponente può essere **negativo** (numeri piccoli) o **positivo** (numeri grandi). Per non memorizzare un segno anche per l'esponente si usa la **rappresentazione in eccesso**: si **somma una costante P** (*bias*) all'esponente, così il valore memorizzato è sempre **positivo**.

- S_M = il segno della mantissa;
- n_M = il numero di bit riservati alla mantissa;
- mantissa = la codifica binaria della mantissa normalizzata;
- n_E = il numero di bit riservati all'esponente;
- esponente = la codifica binaria dell'esponente in eccesso P.



E = esp + P ⟺ esp = E − P

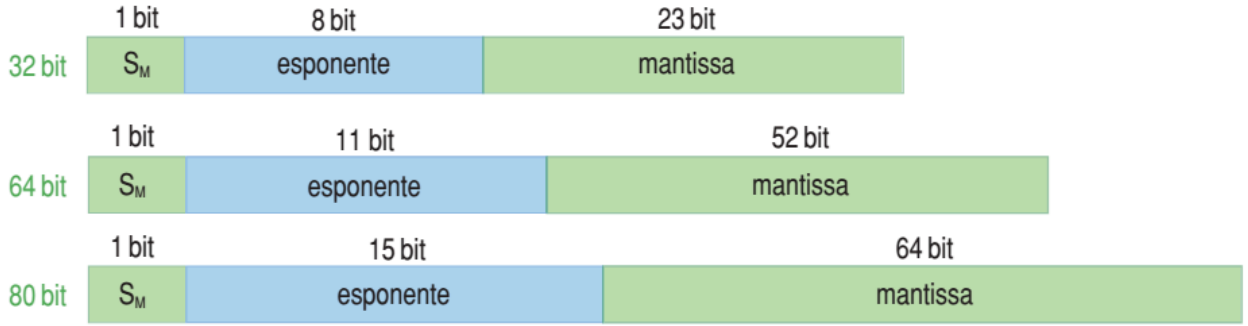
Vantaggio: i numeri si **confrontano** come interi senza segno.

Parte D

Lo standard IEEE-754

Tre precisioni

- **A** singola precisione: 32 bit;
- **B** doppia precisione: 64 bit;
- **C** precisione estrema: 80 bit.



Formato	Totale	Segno	Esponente	Mantissa	Bias
Singola	32 bit	1	8	23	127
Doppia	64 bit	1	11	52	1023
Estesa	80 bit	1	15	64	16383

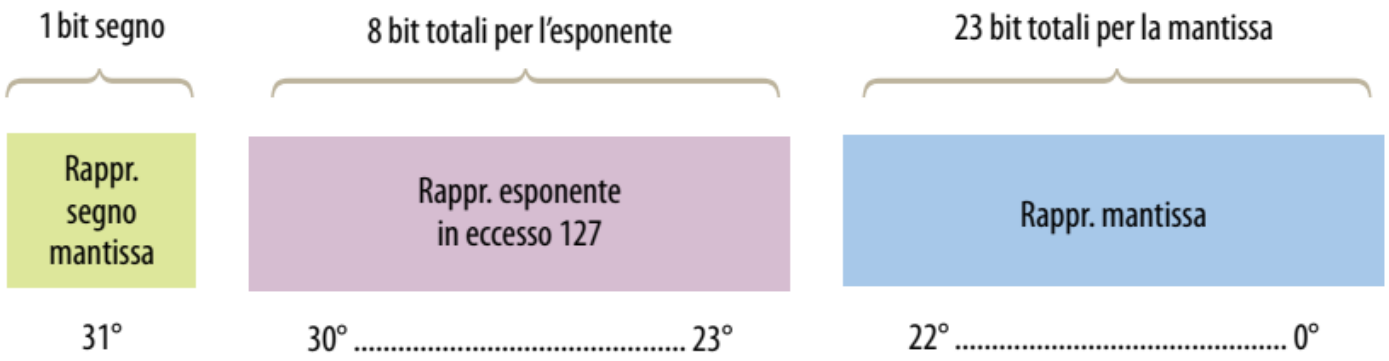
Singola e doppia precisione

Elemento	Singola Precisione	Doppia Precisione
Numero di bit di segno	1	1
Numero di bit esponente	8	11
Numero di bit della frazione F	23	52
Numero di bit, totale	32	64
Rappr. esponente	Eccesso 127	Eccesso 1023
Intervallo esponente	–126 ... +127	–1022 ... +1023

- **float** in C/Java ↔ 32 bit; **double** ↔ 64 bit;
- in Python `float` è un **double** a 64 bit.

La forma dei numeri normalizzati

$$N = 1.xxxxxxxx \cdot 2^{esp}$$



$(-1)^s \cdot 1{,}M \cdot 2^{E-127}$ (con E tra 1 e 254)

Su 32 bit: **bit 31** = S; **bit 30…23** = E (8 bit); **bit 22…0** = M (23 bit).

Codificare il numero 1,0

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB = 1 hidden)
0	0111 1111	000 0000 0000 0000 0000 0000

A questo punto lo scomponiamo in 4 byte:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
0	1000 0000	0000 0000	0000 0000

che in esadecimale viene scritto come **0x3F800000**.

Codifichiamo il numero $V = 1.0_{10}$:
1. il numero è positivo, quindi il bit di segno è uguale a 0;
2. come primo passo codifichiamo il numero in binario: 1.000_2;
3. procediamo con la **normalizzazione**, spostando a destra la virgola di 0 posizioni:
   $V = 1.000 \cdot 2^0$ (**rappresentato senza errori**);
4. codifichiamo ora l'**esponente con eccesso 127**, al quale dobbiamo sommare 0,
   che è il valore ottenuto dalla normalizzazione, $0 + 127 = 127_{10} = 0111\ 1111_2$.

1. S = 0 (positivo);
2. 1,0 = `1,0 × 2⁰` → esp = 0 → E = 0 + 127 = 127 = `0111 1111`;
3. M = 0 (23 zeri).

`0 | 01111111 | 000…0` → **0x3F800000.** 0x3F800000

Codificare 8,5

Quindi otteniamo:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB = 1 hidden)
0	1000 0010	000 1000 0000 0000 0000 0000

a questo punto lo scomponiamo in quattro byte:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
0100 0001	0000 1000	0000 0000	0000 0000

che in esadecimale viene scritto come **0x4108 0000**.

Codifichiamo il numero $\mathbf{V = 8.5_{10}}$:

1. il numero è positivo, quindi il bit di segno è uguale a 0;
2. come primo passo codifichiamo il numero in binario: $\mathbf{1000.1_2}$;
3. ora procediamo con la **normalizzazione**, spostando a destra la virgola di 3 posizioni:
   $V = 1.0001_2 \cdot 2^3$ (**rappresentato senza errori**);
4. codifichiamo ora l'**esponente con eccesso 127**, al quale dobbiamo sommare 3 che è il valore ottenuto dalla normalizzazione, $3 + 127 = 130_{10} = 1000\ 0010_2$.

$8{,}5 = 1000{,}1_2 = 1{,}0001 \times 2^3 \rightarrow E = 3 + 127 = 130 = 1000\ 0010$; $M = 0001\ 000\ldots$

`0 | 10000010 | 00010000000000000000000` → **0x4108 0000.** 0x41080000

Codificare −4,5

Quindi otteniamo:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB = 1 hidden)
1	1000 0001	001 0000 0000 0000 0000 0000

a questo punto lo scomponiamo in quattro byte:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
1100 0000	1001 0000	0000 0000	0000 0000

che in esadecimale viene scritto come **0xC090 0000**.

Codifichiamo il numero $\mathbf{V = -4.5_{10}}$:

1. il numero è negativo, quindi il bit di segno è uguale a 1;
2. come primo passo codifichiamo il numero in binario: $\mathbf{100.1_2}$;
3. ora procediamo con la **normalizzazione**, spostando a destra la virgola di 2 posizioni:
   $V = 1.001_2 \cdot 2^2$ (**rappresentato senza errori**);
4. codifichiamo ora l'**esponente con eccesso 127**, al quale dobbiamo sommare 2 che è il valore ottenuto dalla normalizzazione, $2 + 127 = 129_{10} = 1000\ 0001_2$.

−4,5: S = 1; 4,5 = `100,1₂` = `1,001 × 2²` → E = 129 = `1000 0001`; M = `001 000…`

`1 | 10000001 | 00100000000000000000000` → **0xC090 0000.** 0xC0900000

Codificare −13,75 e 5,875

Codifichiamo il numero **V = −13.75$_{10}$**:

1. il numero è negativo, quindi il bit di segno è uguale a 1;
2. come primo passo codifichiamo il numero in binario: **1101.11$_2$**;
3. ora procediamo con la **normalizzazione**, spostando a destra la virgola di 3 posizioni:
   $V = 1.10111_2 \cdot 2^3$
4. codifichiamo ora l'**esponente con eccesso 127**, al quale dobbiamo sommare 3 che è il valore ottenuto dalla normalizzazione, $3 + 127 = 130_{10} = 1000\ 0010_2$.

Quindi otteniamo:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB = 1 hidden)
1	1000 0010	101 1100 0000 0000 0000 0000

a questo punto lo scomponiamo in quattro byte:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
1100 0001	0101 1100	0000 0000	0000 0000

che in esadecimale viene scritto come **0xC15C0000**.

$-13{,}75 = -1101{,}11_2 = -1{,}10111 \times 2^3 \rightarrow E = 130 \rightarrow$ **0xC15C0000**.

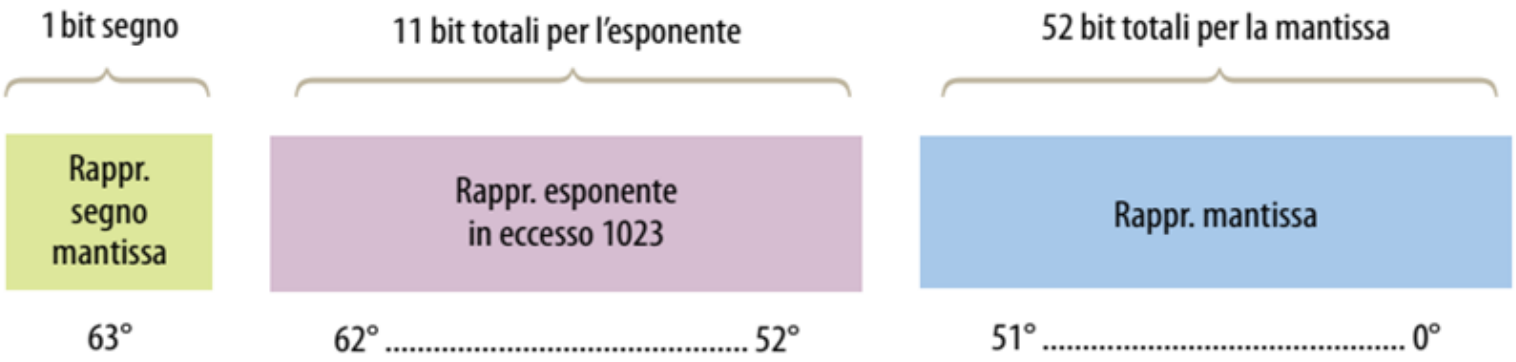
Codifichiamo il numero **V = 5.875$_{10}$**:

1. il numero è positivo, quindi il bit di segno è uguale a 0;
2. come primo passo codifichiamo il numero in binario: **101.111$_2$** (**senza approssimazione**);
3. procediamo con la **normalizzazione**, spostando a destra la virgola di 2 posizioni:
   $V = 1.01111 \cdot 2^2$ (**rappresentato senza errori**);
4. codifichiamo ora l'**esponente con eccesso 127**, al quale dobbiamo sommare 2, che è il valore ottenuto dalla normalizzazione, $2 + 127 = 129_{10} = 1000\ 0001_2$.

Quindi otteniamo:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB = 1 hidden)
0	1000 0001	011 1100 0000 0000 0000 0000

Doppia precisione (64 bit)

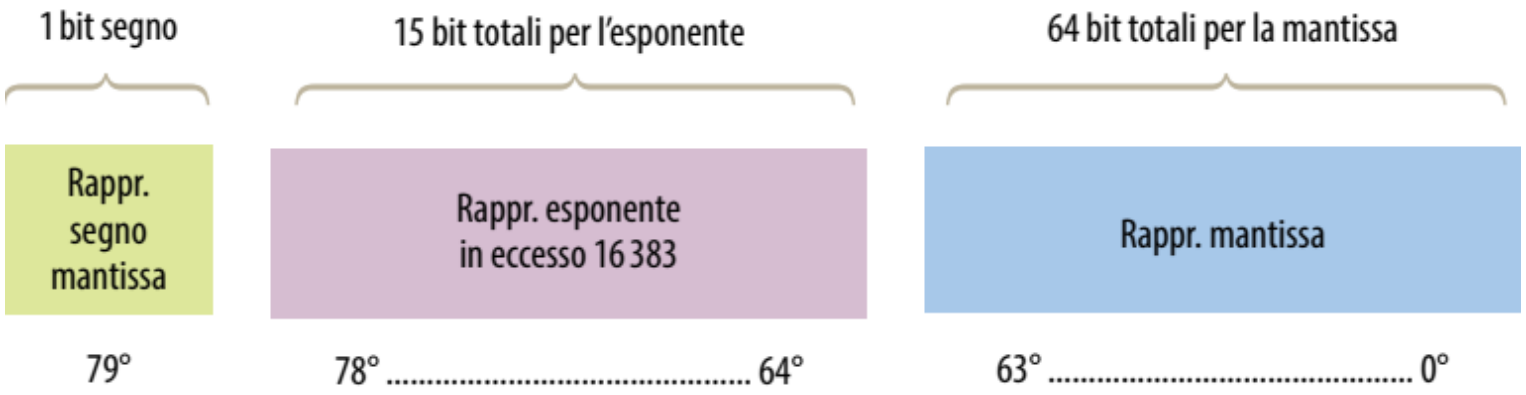


I **64 bit** sono così utilizzati:

- **1 bit** (0 = positivo, 1 = negativo) per il **segno del numero**;
- **11 bit** per l'**esponente** intero **esp**, codificato con **eccesso 1023** ($2^{11-1} - 1$);
- **52 bit** per la codifica della **mantissa m** nell'ordine dal meno significativo al più significativo.

$(-1)^s \cdot 1{,}M \cdot 2^{E-1023}$: 1 bit di segno, **11** di esponente, **52** di mantissa. Precisione: circa **15–16 cifre decimali**.

Precisione estesa (80 bit)



Gli **80 bit** sono così utilizzati:

- **1 bit** (0 = positivo, 1 = negativo) per il **segno del numero**;
- **15 bit** per l'**esponente** intero **esp**, codificato con **eccesso 16383** ($2^{15-1} - 1$);
- **64 bit** per la codifica della **mantissa m** nell'ordine dal meno significativo al più significativo.

Caratteristica: la mantissa di 64 bit contiene **esplicitamente** il bit intero (non c'è hidden bit).

Parte E

Decodifica

Valori limite dell'esponente

Il valore più grande, sapendo che il bias è 127, sarebbe:

$$1111\ 1111_2 - 127_{10} = (2^8 - 1) - 127_{10} = 255 - 127 = 128$$

Nella realtà l'esponente $1111\ 1111_2$ viene riservato per indicare situazioni anomale, e quindi il più grande esponente utilizzabile è 127.

UNDERFLOW

L'**underflow** avviene quando l'esponente è troppo piccolo e non è più rappresentabile con gli 8 bit assegnati.

Con 8 bit E va da 0 a 255, ma **0 e 255 sono riservati** (casi speciali). Per i normalizzati: **E = 1 … 254** → esp = **−126 … +127**.

Minimo esponente

Il valore più piccolo rappresentabile, sempre con bias 127, sarebbe:

$$0000\ 0000_2 - 127_{10} = -127_{10}$$

Anche in questo caso la rappresentazione $0000\ 0000_2$ viene riservata alla rappresentazione dello zero, quindi il più piccolo esponente utilizzabile è −126.

$0 <$ esponente < 255: il numero è normalizzato nella forma $(-1)^{\text{segno}} \cdot (1 + 0.\textit{mantissa}) \cdot 2^{\text{esponente}}$

I casi particolari, che saranno descritti in seguito, hanno rispettivamente:

- esponente = 0 : mantissa denormalizzata;
- esponente = 255 : codifica di ±∞ e codifiche di errore.

Il numero normalizzato più piccolo in singola precisione è $1{,}0 \times 2^{-126} \approx \mathbf{1{,}18 \cdot 10^{-38}}$; il più grande ≈ $\mathbf{3{,}4 \cdot 10^{38}}$.

Procedura di decodifica

Vediamo alcuni esempi con la forma:

$$N = (-1)^{\text{segno}} \cdot (1 + 0.\text{mantissa}) \cdot 2^{\text{esponente}}$$

1. scrivere il numero esadecimale in **binario** (32 bit);
2. separare **S | E (8 bit) | M (23 bit)**;
3. segno: S = 0 → +, S = 1 → −;
4. esponente: **esp = E − 127**;
5. mantissa: **1,M** (si aggiunge l'hidden bit);
6. valore = $\mathbf{(-1)^{s} \cdot 1,M \cdot 2^{esp}}$.

Decodifica: esempio 1

Esempio 1

Decodifichiamo il numero **V = C0400000$_{16}$**.

1. Come primo passaggio lo rappresentiamo in binario:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
1100 0000	0100 0000	0000 0000	0000 0000

2. quindi separiamo **segno**, **esponente** e **mantissa**:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB 1 hidden)
1	1000 0000	100 0000 0000 0000 0000 0000

3. il numero è negativo, dato che il bit di segno è uguale a **1**;
4. il valore della **mantissa** è: 0.1000;

   alla quale aggiungiamo il valore 1 sottinteso → **m** = 1.1_2;
5. il valore dell'**esponente** è: y = 1000 0000$_2$ = 128;

   quindi otteniamo che la normalizzazione ha effettuato uno spostamento di:

   eccesso = 128 – 127 = 1.

   Il numero è quindi il seguente: $n = 1.1_2 \cdot 2^{1.}$;
6. da esso moltiplicando per 2^1 ricaviamo la notazione binaria e il numero decimale:

   $n = 11_2 = 3_{10}$

   al quale ricordiamo di aggiungere il segno negativo al risultato finale: **$n = -3_{10}$**.

0xC0400000 → S = 1; E = `1000 0000` = 128 → esp = 1; M = `100…` → $1{,}1_2 = 1{,}5$ → $-1{,}5 \cdot 2 = -3$.

Decodifica: esempio 2

Esempio 2

Decodifichiamo il numero **V = 3F40 0000$_{32}$**.

1. Come primo passaggio lo rappresentiamo in binario:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
0011 1111	0100 0000	0000 0000	0000 0000

2. quindi separiamo **segno**, **esponente** e **mantissa**:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit 1 hidden)
0	0111 1110	100 0000 0000 0000 0000 0000

3. il numero è negativo, dato che il bit di segno è uguale a 1;
4. il valore della **mantissa** è: 0.1000;
   alla quale aggiungiamo il valore 1 sottinteso → **m** = 1.1_2;
5. il valore dell'**esponente** è: y = 0111 1110$_2$ = 126;
   quindi otteniamo che la normalizzazione ha effettuato uno spostamento di:
   eccesso = 126 – 127 = –1.
   Il numero è quindi il seguente: $n = 1.1_2 \cdot 2^{-1}$;
6. da esso motliplicando per 2^{-1} ricaviamo la notazione binaria e il numero decimale:
   $n = 0.11_2 = 2^{-1} + 2^{-2} = 0.5 + 0.25 = \mathbf{0.75}_{10}$.

0x3F400000 → S = 0; E = 126 → esp = −1; $1{,}1_2 = 1{,}5$ → $1{,}5 \cdot 2^{-1} =$ **+0,75**.

Nota di correzione: nel testo dell'immagine si dice "il numero è negativo", ma il bit di segno è **0**, quindi il numero è **positivo**: +0,75.

Decodifica: esempio 3

Esempio 3

Decodifichiamo il numero $V = BF00\ 0000_{32}$.

1. Come primo passaggio lo rappresentiamo in binario:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
0011 1111	0000 0000	0000 0000	0000 0000

2. quindi separiamo segno, esponente e mantissa:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit 1 hidden)
0	0111 1110	000 0000 0000 0000 0000 0000

3. il numero è negativo dato che il bit di segno è uguale a 1;
4. il valore della mantissa è: 0.0; alla quale aggiungiamo il valore 1 sottinteso → $m = 1.0_2$;
5. il valore dell'esponente è: $y = 0111\ 1110_2 = 126$;
   quindi otteniamo che la normalizzazione ha effettuato uno spostamento di:
   eccesso = 126 – 127 = –1.

   Il numero è quindi il seguente: $n = 1.0 \cdot 2^{-1}$;

Nota di correzione: il numero è indicato come BF000000, ma il primo byte è mostrato come `0011 1111` (cioè 3F). Con 3F000000 si ha S = 0, E = 126 → **+0,5**; con **0xBF000000** (primo byte `1011 1111`) si ottiene **–0,5**.

Decodifica: esempio 4

Esempio 4

Decodifichiamo il numero **V = BF61 0000$_{32}$**.

1. Come primo passaggio lo rappresentiamo in binario:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
1011 1111	0111 0001	0000 0000	0000 0000

2. quindi separiamo **segno**, **esponente** e **mantissa**:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit 1 hidden)
1	0111 1110	111 0001 0000 0000 0000 0000

3. il numero è negativo dato che il bit di segno è uguale a 1;
4. il valore della **mantissa** è: 0. **110 0001**; alla quale aggiungiamo il valore 1 sottinteso
   → **m** = 1. **110 0001**$_2$;
5. il valore dell'**esponente** è: y = 0111 1110$_2$ = 126;
   quindi otteniamo che la normalizzazione ha effettuato uno spostamento di:
   eccesso = 126 – 127 = –1.
   Il numero è quindi il seguente: $n = 1.110\,0001_2 \cdot 2^{-1}$;
6. da esso moltiplicando per 2^{-1} ricaviamo la notazione binaria e il numero decimale:
   $n = 0.1110\,0001_2 =$
   $= 2^{-1} + 2^{-2} + 2^{-3} + 2^{-4} + 2^{-8} = 0.5 + 0.25 + 0.125 + 0.00390625 = 0.87890625_{10}$;
   che deve essere cambiato di segno ottenendo: **n = –0.87890625$_{10}$**.

0xBF610000 → S = 1; E = 126 → esp = −1; M = `110 0001…` → $1{,}1100001_2 = 1{,}7578125$ → **−0,87890625**.

Decodifica: esempio 5

Esempio 5

Decodifichiamo il numero $V = 3EA8\ 0000_{32}$.

1. Come primo passaggio lo rappresentiamo in binario:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
0011 1110	1010 1000	0000 0000	0000 0000

2. quindi separiamo segno, esponente e mantissa:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit 1 hidden)
0	0111 1101	010 1000 0000 0000 0000 0000

3. il numero è positivo dato che il bit di segno è uguale a 0;

4. il valore della mantissa è: 0.010 1000; alla quale aggiungiamo il valore 1 sottinteso

$\rightarrow m = 1.0101_2$;

5. il valore dell'esponente è: y = 0111 1101 = 125;
quindi otteniamo che la normalizzazione ha effettuato uno spostamento di:
eccesso = 125 – 127 = –2.
Il numero è quindi il seguente: $n = 1.0101_2 \cdot 2^{-2}$;

6. da esso moltiplicando per 2^{-2} ricaviamo la notazione binaria e il numero decimale:
$n = 0.10101_2 = 2^{-1} + 2^{-3} + 2^{-5} = 0.328125_{10}$ che non deve essere cambiato di segno.

0x3EA80000 → S = 0; E = `0111 1101` = 125 → esp = −2; M = `010 1000…` → $1{,}0101_2 = 1{,}3125$ → $1{,}3125 \cdot 2^{-2} = 0{,}328125$.

Parte F

Casi speciali e denormalizzati

I numeri più piccoli

$$1.0 \cdot 2^{-126}$$

in quanto la mantissa normalizzata deve avere valore 1 prima della virgola.
Togliendo questo vincolo il numero più piccolo sarebbe

$$0.000000\ldots1$$

cioè un numero con il valore 1 come ultimo bit, quindi:

$$1 \cdot 2^{-23}$$

Perciò il numero più piccolo rappresentabile diviene:

$$1.0 \cdot 2^{-126} \cdot 2^{-23} = 2^{-149}$$

Nella notazione IEEE-P754 a 32 bit il numero più piccolo

$$1.0 \cdot 2^{-126}$$

Quando si vuole "denormalizzare" la mantissa, convenzionalmente si utilizza la configurazione di **tutti zeri per l'esponente**. Quindi il numero più piccolo risulta essere così codificato:

SEGNO	ESPONENTE	MANTISSA DENORMALIZZATA
0	0000 0000	000 0000 0000 0000 0000 0001

che in esadecimale viene scritto come **$0x00000001_{16}$**.

Tipo	Valore (singola)
Normalizzato minimo	$2^{-126} \approx 1{,}18 \cdot 10^{-38}$
Denormalizzato minimo	$2^{-149} \approx 1{,}4 \cdot 10^{-45}$

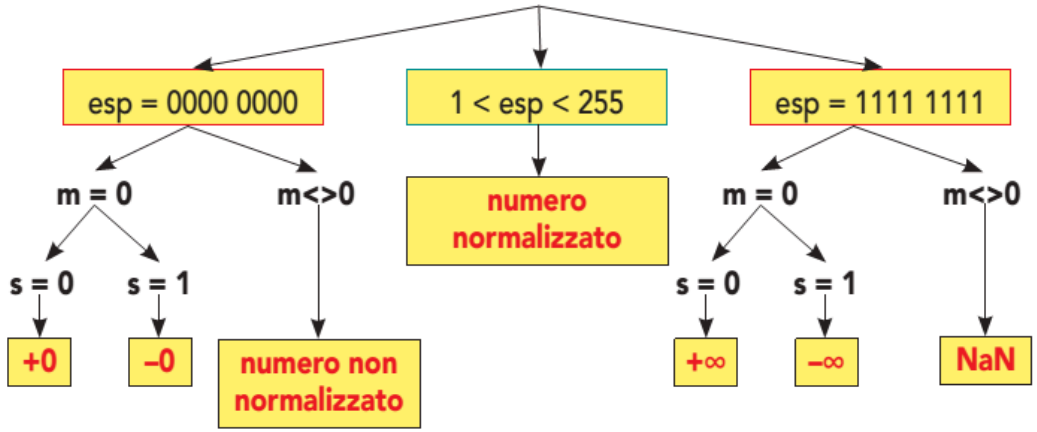
Sotto 2^{-149} il numero viene approssimato a 0: **underflow**.

Casi particolari: schema

Anche la codifica con l'esponente con tutti i bit a 1 (esp = 1111.1111) non viene interpretato come numero normalizzato ma permette di rappresentare:

- con m = 0 e s = 0, convenzionalmente il +∞;
- con m = 0 e s = 1 e m = 0, convenzionalmente il –∞;
- con m<>0, un insieme di 2^{23} configurazioni considerate Not a Number (NaN), che sono utilizzate per applicazioni particolari.

La figura seguente schematizza tutte le rappresentazioni possibili in formato IEEE-754:



E	M	Significato
0	0	±0 (a seconda di S)
0	≠ 0	denormalizzato
1…254	qualsiasi	normalizzato

Denormalizzati: il range

> Il valore di **esponente** = 0000 0000 permette di codificare quindi un sottoinsieme di numeri molto piccoli, che hanno un range da 0 a 2^{-149}.

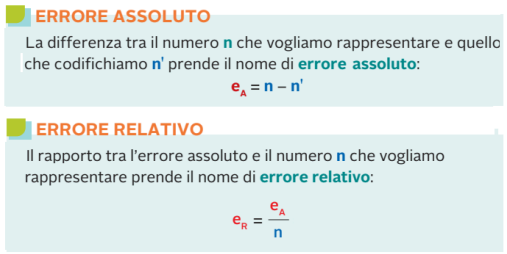
Con E = 0 non c'è hidden bit: il valore è $(-1)^s \cdot 0{,}M \cdot 2^{-126}$. Riempiono il vuoto tra 0 e il più piccolo normalizzato (**underflow graduale**).

```
import struct
print(struct.unpack(">f", bytes.fromhex("00000001"))[0])  # 1.4e-45
print(struct.unpack(">f", bytes.fromhex("7F800000"))[0])  # inf
```

Parte G

Errori di rappresentazione

Errore assoluto e relativo



- **errore assoluto**: E_a = |valore vero − valore rappresentato|;
- **errore relativo**: E_r = E_a / |valore vero|.

> Nelle notazioni non normalizzate l'**errore relativo massimo** non è costante.

L'errore **assoluto** cresce con la grandezza del numero; l'errore **relativo** resta **circa costante** ($\approx 2^{-24}$ in singola precisione).

Il numero 0,1 non è esatto!

$$V = .1_{10}$$

ESEMPIO

Codifichiamo il numero $V = .1_{10}$.
La codifica binaria è: $V = .1_{10} = .0001101011001100110011...._2 = .0(0011)_2$;
dopo aver normalizzato la mantissa otteniamo:
$V = (1.1001) \cdot 2^{-4} = 1.1001\ 1001\ 1001\ 1001\ 1001\ 1001\ ...\ 1001 \cdot 2^{-4}$;

Dato che il numero è periodico, la mantissa non potrà mai essere completamente rappresentata, quindi si introduce un errore di approssimazione.

1.	.1 × 2 = 0.2 riporto **0**
2.	.2 × 2 = 0.4 riporto **0**
3.	.4 × 2 = 0.8 riporto 0
4.	.8 × 2 = 1.6 riporto **1**
5.	.6 × 2 = 1.2 riporto **1**
6.	.2 × 2 = 0.4 riporto 0
7.	...

Troncare o arrotondare

$$V = .(1100) * 2^{-3} = .1100\ 1100\ 1100\ 1100\ 1100\ 1100\ 1100\ 1100... * 2^{-3}$$

Possiamo scegliere tra due opzioni alternative:

- **troncare** la mantissa all'n-esima cifra richiesta, quindi si escludono le cifre in eccesso:
  m = .1001 1001 1001 1001 1001 100;
- **arrotondare** la mantissa aggiungendo 1 all'ultima cifra che si considera:
  m = .1001 1001 1001 1001 1001 101.

- **troncamento**: si scartano i bit in eccesso;
- **arrotondamento**: se il primo bit scartato è 1 si **somma 1** all'ultimo bit tenuto.

> Nei **linguaggi C** e **Java** si è optato per la seconda per la seconda soluzione, cioè di **arrotondare la mantissa**.

I linguaggi C e Java **arrotondano**.

Esempio: 0,15

Esempio 1
Rappresentiamo **V = .15$_{10}$**.
La **codifica binaria** è la seguente: V = .15$_{10}$ = .00(1001)$_2$.
La **normalizzazione** trasforma il numero in: V = (1.001) · 2^{-3} = 1.001(1001) · 2^{-3}.
Con un **esponente in eccesso 127** è: 127 − 3 = 124$_{10}$ = **0111 1100**$_2$.

1. Lo codifichiamo con **troncamento**:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB = 1 hidden)
0	0111 1100	0011 0011 0011 0011 0011 001

Lo scomponiamo in 4 byte:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
0011 1110	0001 1001	1001 1001	1001 1001

e lo scriviamo in esadecimale come 0x3E199999.

2. Lo codifichiamo con **arrotondamento**:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit 1 hidden)
0	0111 1100	0011 0011 0011 0011 0011 010

Lo scomponiamo in 4 byte:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
0011 1110	0001 1001	1001 1001	1001 1010

e lo scriviamo in esadecimale come 0x3E19999A.

> Come si può vedere, per ottenere la **rappresentazione con arrotondamento** basta sommare 1 alla rappresentazione con troncamento.

Per 0,15: **troncamento** → `0x3E199999`; **arrotondamento** → **0x3E19999A**.

Esempio: −1/3

Esempio 2

Rappresentiamo $V = -1/3_{10}$.
La **codifica binaria** è la seguente: $V = -0.(3)_{10} = .(0101)_{2}$.
La **normalizzazione** trasforma il numero in: $V = (.0101)_{2} \cdot 2^{-2} = 1.(01) \cdot 2^{-2}$.
Con un **esponente in eccesso 127** è: $127 - 2 = 125_{10} = 0111\ 1101_{2}$.

1. Lo codifichiamo con **troncamento**:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB = 1 hidden)
1	0111 1101	010 1010 1010 1010 1010 1010

Lo scomponiamo in 4 byte:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
1011 1110	1010 1010	1010 1010	1010 1010

e lo scriviamo in esadecimale come 0xBEAAAAAA.

2. Lo codifichiamo con **arrotondamento**:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB = 1 hidden)
1	0111 1101	010 1010 1010 1010 1010 1011

Lo scomponiamo in 4 byte:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
1011 1110	1010 1010	1010 1010	1010 1011

e lo scriviamo in esadecimale come 0xBEAAAAAB.

$-1/3 = -0{,}010101\ldots_2$ (periodico). Arrotondato in singola: **0xBEAAAAAB**; troncato: 0xBEAAAAAA.

Esempio: 12,6

Esempio 3

Come ultimo esempio rappresentiamo **V = 12.6$_{10}$**.
La **codifica binaria** è la seguente: V = 12.6 = 1100.(1001)$_2$.
La **normalizzazione** trasforma il numero in: V = 1100.(1001)$_2$ = 1.**100**(1001) · 2^3.
Con un **esponente in eccesso 127** è: 127 + 3 = 130_{10} = **1000 0010**$_2$.

1. Lo codifichiamo con **troncamento**:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB = 1 hidden)
0	1000 0010	100 1001 1001 1001 1001 1010

1. Lo codifichiamo con **troncamento**:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB = 1 hidden)
0	1000 0010	100 1001 1001 1001 1001 1010

Lo scomponiamo in 4 byte:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
0100 0001	0100 1001	1001 1001	1001 1001

e lo scriviamo in esadecimale come 0x41499999.

2. Lo codifichiamo con **arrotondamento**:

SEGNO	ESPONENTE	MANTISSA NORMALIZZATA (23 bit MSB = 1 hidden)
0	1000 0010	100 1001 1001 1001 1001 1010

Lo scomponiamo in 4 byte:

BYTE 1	BYTE 2	BYTE 3	BYTE 4
0100 0001	0100 1001	1001 1001	1001 1010

e lo scriviamo in esadecimale come 0x4149999A.

12,6 = `1100,1001 1001…₂` = `1,1001 0011 0011… × 2³`

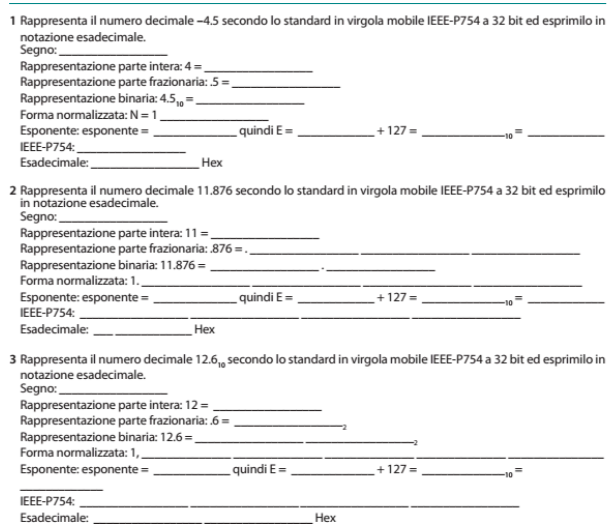
- troncamento: 0x41499999;
- arrotondamento: 0x4149999A

Riepilogo: i punti chiave

- **Virgola mobile:** $N = (-1)^s \cdot 1{,}M \cdot 2^{esp}$.
- **Normalizzare:** mantissa del tipo 1,xxx; **hidden bit** non memorizzato.
- **Esponente in eccesso:** E = esp + bias (127 / 1023).
- **IEEE-754:** 32 = 1+8+23; 64 = 1+11+52; 80 = 1+15+64.
- **Casi speciali:** E=0 → ±0 / denormalizzati; E=255 → ±∞ / NaN.
- **Errori:** i numeri come 0,1 non sono esatti; si **tronca** o si **arrotonda**.
- **Errore relativo** ≈ costante; **assoluto** cresce con l'ordine di grandezza.

VERIFICA... le competenze

Verifica: esercizi



1 Rappresenta il numero decimale –4.5 secondo lo standard in virgola mobile IEEE-P754 a 32 bit ed esprimilo in notazione esadecimale.
Segno: _____
Rappresentazione parte intera: 4 = _____
Rappresentazione parte frazionaria: .5 = _____
Rappresentazione binaria: 4.5_{10} = _____
Forma normalizzata: N = 1 _____
Esponente: esponente = _____ quindi E = _____ + 127 = _____$_{10}$ = _____
IEEE-P754: _____
Esadecimale: _____ Hex

2 Rappresenta il numero decimale 11.876 secondo lo standard in virgola mobile IEEE-P754 a 32 bit ed esprimilo in notazione esadecimale.
Segno: _____
Rappresentazione parte intera: 11 = _____
Rappresentazione parte frazionaria: .876 = . _____ _____ _____
Rappresentazione binaria: 11.876 = _____ . _____
Forma normalizzata: 1. _____ _____ _____ _____
Esponente: esponente = _____ quindi E = _____ + 127 = _____$_{10}$ = _____
IEEE-P754: _____ _____ _____ _____
Esadecimale: ___ _____ Hex

3 Rappresenta il numero decimale 12.6_{10} secondo lo standard in virgola mobile IEEE-P754 a 32 bit ed esprimilo in notazione esadecimale.
Segno: _____
Rappresentazione parte intera: 12 = _____
Rappresentazione parte frazionaria: .6 = _____$_2$
Rappresentazione binaria: 12.6 = _____ _____$_2$
Forma normalizzata: 1, _____ _____ _____ _____
Esponente: esponente = _____ quindi E = _____ + 127 = _____$_{10}$ = _____
IEEE-P754: _____ _____ _____ _____
Esadecimale: _____ _____ Hex

Esercizi da svolgere in singola precisione:

1. Codifica **−4,5** in esadecimale.
2. Codifica **11,876** con troncamento e con arrotondamento.
3. Codifica **12,6** con troncamento e con arrotondamento.

Verifica: soluzioni

1. −4,5: **0xC0900000** (S=1, E=129, M=`001…`).
2. 11,876 = `1,01111000000100000011001… × 2³` (E = 130): troncamento **0x413E0418**; arrotondamento **0x413E0419**.
3. 12,6: troncamento **0x41499999**; arrotondamento **0x4149999A**.
4. 0x40490FDB → S=0; E=`1000 0000`=128 → esp=1; 1,5707964 · 2 = **3,1415927** (π).

Verifica in Python:

```
import struct
for x in (-4.5, 11.876, 12.6): print(struct.pack(">f", x).hex().upper())
print(struct.unpack(">f", bytes.fromhex("40490FDB"))[0])
```

Grazie per l'attenzione

Domande?